

Algorithm Lab (CS2271)

Assignment 5

Department of Computer Science and Technology
Indian Institute of Engineering Science and Technology, Shibpur

Q1. Problem Statement

Let $S = \{0, 1, 2, \dots, n-1\}$ be a set of n distinct elements. Initially, each element forms its own separate set, so the collection consists of n disjoint singleton sets. This initial configuration is represented using a parent array P of size n , where $P[i] = i$ for all $0 \leq i \leq n-1$, and a rank array R , where $R[i] = 0$ for all $0 \leq i \leq n-1$.

Design and implement an efficient **Disjoint Set (Union-Find)** data structure that supports two fundamental operations: $Find(x)$, which returns the representative (root) of the set containing element x , and $Union(x, y)$, which merges the two sets containing elements x and y . To improve efficiency, apply **path compression** during the $Find$ operation so that each visited node directly points to the root, and apply **union by rank** during the $Union$ operation so that the tree with smaller height is attached under the tree with larger height, thereby keeping the tree shallow.

Q2. Problem Statement

In a large school campus, thousands of students dynamically form different friend groups, where new friendships are created daily and teachers frequently need to verify whether two students belong to the same group. Initially, each student forms an independent group. As interactions increase, multiple groups gradually merge, and it becomes essential to efficiently manage these dynamic group connections.

Given n students and m randomly generated friendship operations, design and implement an optimized **Disjoint Set (Union-Find)** data structure using **dynamic arrays**, which supports two fundamental operations:

- **Union**(x, y): Merges the sets containing students x and y .
- **Find**(x): Returns the representative (leader) of the set containing student x .

Let the initial parent array be defined as:

$$P[i] = i, \quad \text{for all } 0 \leq i \leq n-1$$

To enhance performance, apply **path compression** during the $Find$ operation and **union by rank** during the $Union$ operation. The system must efficiently process all m operations, ensuring fast connectivity queries even for large values of n and m .

Q3. Problem Statement

A city contains N buildings, initially all isolated, which can be mathematically modeled as N disjoint singleton sets, where the parent array is initialized as

$$P[i] = i, \quad \text{for all } 0 \leq i \leq N - 1.$$

Over time, M road construction operations are performed, where each operation connects two buildings (u, v) , with $0 \leq u, v \leq N - 1$. To efficiently manage these dynamic connections, design a **Disjoint Set (Union-Find)** data structure supporting two operations: $Union(u, v)$, which merges the sets containing u and v , and $Find(x)$, which returns the representative (leader) of the set containing x . After each union operation, the system must verify whether the entire city has become fully connected, which mathematically implies that all buildings belong to a single connected component, i.e., there exists a unique representative r such that

$$Find(i) = r, \quad \text{for all } 0 \leq i \leq N - 1.$$